# TokenOptimizer

Project Documentation

Token-optimized Claude automation — shrink text before it reaches the model, fully offline when needed.

Version 0.1.0 • Generated 15 Jul 2026

# Contents

# 1. Introduction & Purpose

TokenOptimizer reduces the number of tokens sent to a Large Language Model, cutting cost and latency per call while preserving every fact a downstream agent needs. It points at a document, a JIRA issue, or a GitHub pull request, shrinks the text through a layered pipeline, and reports exactly how many tokens were saved.

The central design goal is to **reduce LLM usage**. Because of that, the optimizer works **fully offline with no API key** — the deterministic and extractive strategies never call a cloud model. When a Claude API key or a local model is available, quality improves further, but it is never required.

# 2. Key Capabilities

- Layered text reduction: deterministic cleanup, structure-aware extraction, and summarization.
- Works fully offline — no API key needed for real token savings.
- Three summarization tiers chosen automatically: Claude Haiku → local Ollama model → entity-anchored extractive.
- Accurate token counting via API, tiktoken (offline BPE), or a heuristic fallback.
- Desktop UI and command-line interface, plus a Windows launcher (run.bat).
- Detailed per-run logging with no secret leakage.
- Integrations: JIRA, GitHub, Jenkins, GitLab, Azure DevOps.

# 3. Architecture Overview

Input text flows through the pipeline in `optimize/text_pipeline.py`. Each stage is skip-if-noop and only recorded when it actually changes the text:

```
raw text → deterministic reductions → summarization (optional tier) →
whitespace/dedupe compression → optimized text + token report
```

- **Sources** (integrations/) turn documents/JIRA/GitHub into text.
- **optimize/local.py** — deterministic offline reductions.
- **optimize/local_model.py** — optional local Ollama summarizer.
- **optimize/tokens.py** — offline token counting.
- **run_log.py** — writes a full record of every run.

# 4. Optimization Strategies

Deterministic offline stages (always run; no API, no model):

| Stage | What it does |
|---|---|
| unicode | Folds fancy Unicode to ASCII; strips zero-width chars and emoji. |
| framing | Removes conversational/LLM packaging (preambles, sign-offs, follow-up offers). |
| boilerplate | Drops page numbers, headers/footers, rules, TOC dot leaders. |
| fields | Collapses 'Label:'/value blocks into tight 'label: value'; drops placeholder fields. |

| Stage | What it does |
|---|---|
| punctuation | Collapses decorative/repeated punctuation and bullet glyphs. |
| filler | Substitutes verbose phrases with concise equivalents; deletes empty filler. |
| dedupe-paragraphs | Removes near-duplicate paragraphs (normalized fingerprint). |
| compress | Collapses whitespace, dedupes repeated lines, head/tail-truncates. |

## 5. Summarization Tiers

Enabled with `--summarize` (or the UI checkbox). The best available tier is chosen automatically:

| Tier | Needs | Quality |
|---|---|---|
| Claude Haiku | ANTHROPIC_API_KEY | Abstractive rewrite; highest quality. |
| Local Ollama model | TOKENOPT_LOCAL_MODEL + running Ollama | Abstractive, zero cloud tokens. |
| Entity-anchored extractive | Nothing (pure offline) | Keeps top sentences; never drops error codes, versions, IDs, paths, URLs. |

## 6. Token Counting

| Backend | When used | Reported as |
|---|---|---|
| API count_tokens | API key configured | api |
| tiktoken (cl100k_base) | tiktoken installed, no key | tiktoken |
| char/word heuristic | fallback | estimate |

## 7. Minimal Prompt Request

`build_prompt_request()` assembles the smallest reasonable Messages-API request: a cached system prefix kept separate (so prompt caching stays hot), a terse task line, and the optimized data under one compact <data> delimiter — no repeated instructions, no padding.

## 8. Input Sources

- Documents — .docx (python-docx), .txt, .md, .log.
- JIRA — fetch issues by JQL query.
- GitHub — a single PR or all open PRs (optionally with diffs).
- Jenkins / GitLab / Azure DevOps — integration clients.
- Credentials come from the UI form or fall back to environment / .env.

# 9. Interfaces

**Desktop UI** (`tokenopt ui`): Tkinter app with Document / JIRA / GitHub tabs. Auto-optimizes on load; shows Original vs Optimized panes and a token-review panel (Original / Optimized / Saved % / Counted-via).

**Command line**:

| Command | Purpose |
|---|---|
| optimize-doc --file F [--summarize] | Optimize a document. |
| ui | Launch the desktop UI. |
| triage-jira --jql Q [--max N] | Triage JIRA issues. |
| triage-jenkins --job J [--build B] | Root-cause a Jenkins failure. |
| review-github-pr --owner O --repo R --number N | Review a pull request. |
| triage-github-prs --owner O --repo R [--diffs] | Triage all open PRs. |
| demo | Run on canned data. |

**Windows launcher**: `run.bat` — double-click to open the UI, or `run.bat <command>` from a terminal. Uses the project venv directly; no activation needed.

# 10. Per-Run Logging

Every run writes one timestamped file to `logs/` (set via TOKENOPT_LOG_DIR). Each file has a human-readable report and a machine-parseable JSON block. Captured details:

- Run ID, timestamp, command, status (ok / error).
- Source (file / JQL / PR) and resolved options.
- Config snapshot — secrets reduced to booleans (api_key_configured: true), never their values.
- Metrics: raw→optimized tokens, saved, reduction %, token_method, summary_tier, stages, char counts, cost, duration_ms.
- Environment: app version, Python version, platform.
- Output path and any error message.

Logging never raises — a logging failure cannot break a run — and `logs/` is git-ignored.

# 11. Configuration

| Variable | Default | Purpose |
|---|---|---|
| ANTHROPIC_API_KEY | — | Enables Haiku summarization + exact token counts. |
| TOKENOPT_MODEL | claude-opus-4-8 | Main reasoning model. |
| TOKENOPT_SUMMARY_MODEL | claude-haiku-4-5 | Cheap summarization model. |
| TOKENOPT_LOCAL_MODEL | — | Ollama model for offline abstractive summary. |

| Variable | Default | Purpose |
|---|---|---|
| TOKENOPT_LOCAL_MODEL_URL | localhost:11434 | Ollama server URL. |
| TOKENOPT_CACHE_DIR | .tokenopt_cache | Response cache location. |
| TOKENOPT_LOG_DIR | logs | Per-run log location. |

## 12. Testing & Reliability

- Offline test suite (pytest tests/) covers every strategy — no network or API key required.
- Graceful degradation: no key → offline; no tiktoken → heuristic; Ollama down → extractive.
- Disk response cache serves repeated identical calls.
- No secret logging — verified by a dedicated test.

## 13. Security Notes

- Keep real API keys only in .env (git-ignored), never in .env.example.
- Secrets never appear in run logs or optimized output.
- If a key is ever committed, revoke it immediately — history rewrites do not undo exposure.

## 14. Getting Started

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -e .
.venv\Scripts\python.exe -m pip install tiktoken # optional: exact offline
counts

run.bat # open the UI
run.bat optimize-doc --file report.docx # optimize a document
run.bat optimize-doc --file notes.txt --summarize
```

The offline optimizer needs no API key — pick a document in the UI or use `optimize-doc` and the token savings appear immediately. A detailed log is written to `logs/` for every run.